**Full Stack Assignment – 46**

**Assignment ID:** FSD-46
**Title:** Rate-Limited API Access Management System
**Duration:** 3 Hours

## 1. Objective

Design and develop a **Rate-Limited API Access Management System** that restricts how frequently users can invoke protected APIs.
The system must enforce rate limits accurately and ensure **data persistence using a database**.

## 2. User Roles

**USER**

- Register and log in to the system
- Access protected APIs
- View own API usage statistics

**ADMIN**

- Register and log in to the system
- Define and manage rate limits
- View API usage across all users

## 3. Functional Requirements

### Authentication & Authorization

- Implement user registration and login
- Use JWT-based authentication
- Protect all backend APIs using JWT
- Enforce role-based access control on APIs and frontend views

### Rate Limit Configuration

- ADMIN can define rate limits (example: X requests per Y minutes)
- Rate limit rules must be stored in the database
- Rate limit rules must be retrievable dynamically

### API Usage Tracking

- Each API request must be tracked per user
- Usage data must be stored in the database
- Rate limit checks must occur before request processing

### Rate Limit Enforcement

- Requests exceeding the allowed rate must be rejected
- Rejected requests must return a proper error response
- Rate limit counters must reset according to the configured window

**4. Business Rule (Critical)**

A user must not be allowed to access protected APIs once the defined rate limit is exceeded within the configured time window.
All excess requests must be blocked until the rate limit window resets.

**5. Technical Constraints (Mandatory)**

- All rate limit and usage data must persist in a database
- In-memory or hardcoded rate limit counters are not allowed
- Backend APIs must reflect actual database state
- JWT authentication and role validation are mandatory
- Use environment variables for sensitive configuration

**6. Frontend Requirements**

- Login and registration pages
- Role-based dashboards for USER and ADMIN
- API usage statistics view for USER
- Rate limit configuration and usage overview for ADMIN

UI design is not evaluated; correctness of rate-limiting logic is the priority.

**7. Submission Requirements (Strict)**

**GitHub Repository**

- Backend source code
- Frontend source code
- Proper folder structure
- Clear and meaningful commit history

**Live Deployment**

- Live frontend deployment link
- Live backend deployment link
- Backend must be connected to the database

**README.md (Mandatory)**

The README file must include:

- Project overview
- Tech stack used
- User roles and permissions

- API endpoints
- Database schema or collections
- Steps to run the project locally
- Live frontend and backend links

**8. Evaluation Note**

Evaluation focuses on **rate limit enforcement accuracy, database-backed tracking, role control, and end-to-end execution**.